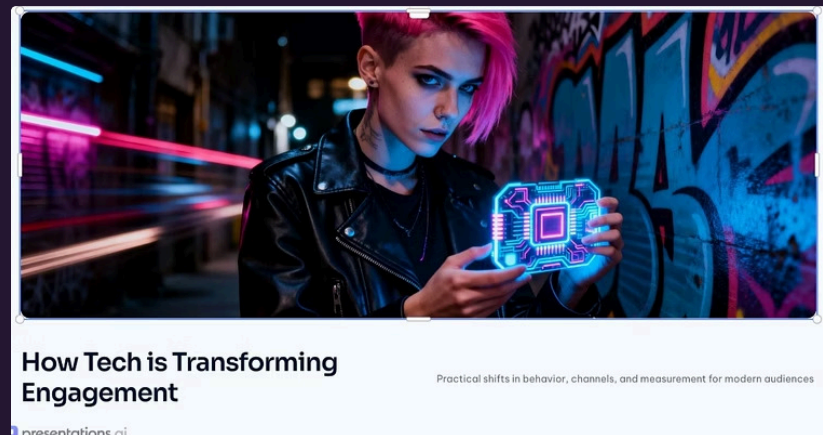




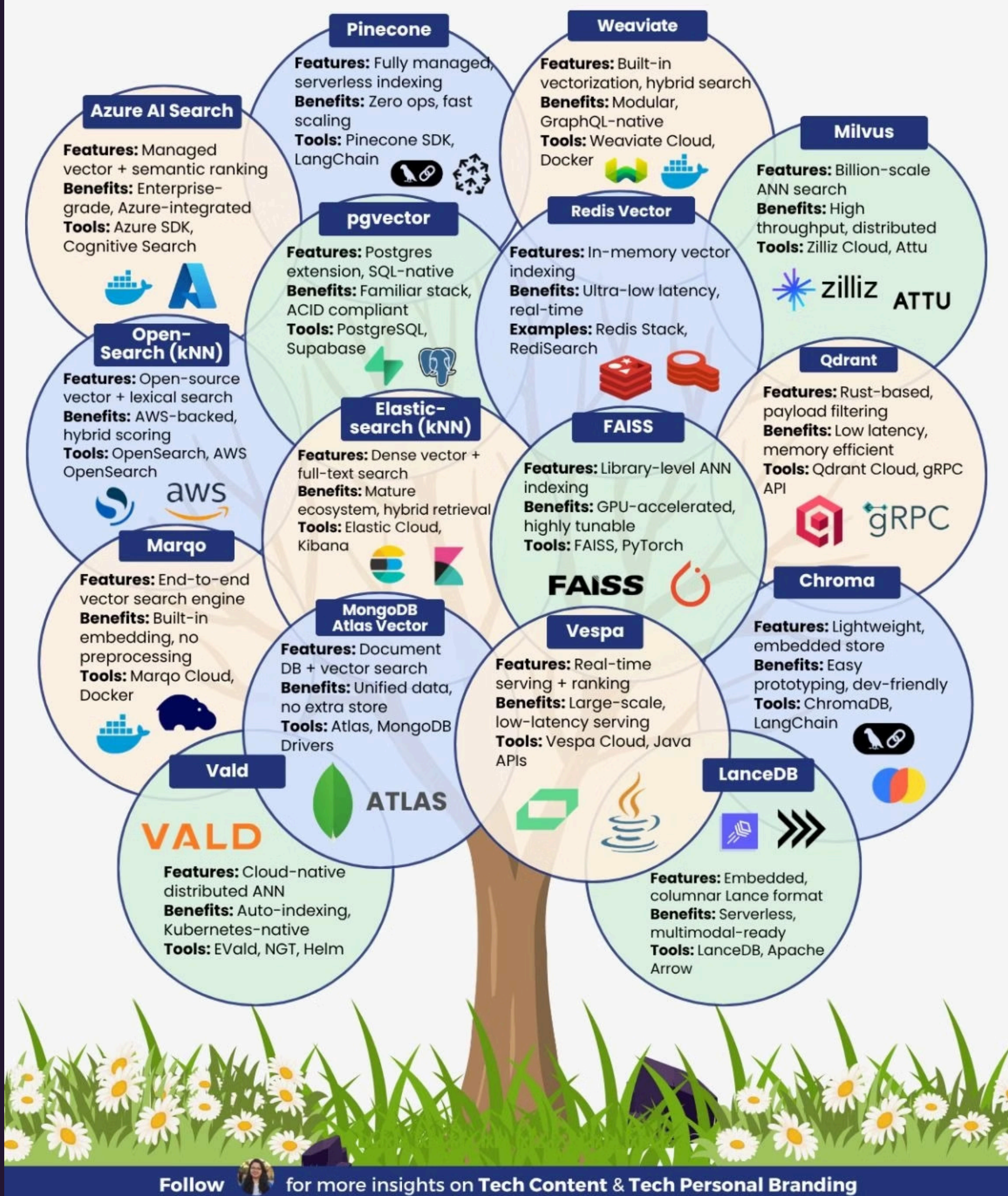
The Intelligent Candidate Discovery & Ranking Engine

Companies miss thousands of brilliant candidates every year — not because they aren't there, but because legacy tools can't see them. It's time to move beyond Ctrl+F recruitment.

AI-POWERED RECRUITMENT

SYSTEM DESIGN

16 Types of Vector Databases



Popular vector databases:

- **Pinecone** → Fully managed, production-ready scaling
- **Weaviate** → Hybrid search + built-in vectorization
- **Milvus** → Designed for billion-scale vector search
- **Qdrant** → Fast, lightweight, efficient
- **Chroma** → Simple choice for prototyping
- **pgvector** → Vector search inside PostgreSQL
- **Redis Vector** → Ultra-low latency retrieval
- **FAISS** → High-performance library from Meta
- **MongoDB Atlas Vector Search** → Built into MongoDB
- **OpenSearch / Elasticsearch** → Hybrid keyword + vector search
- **LanceDB** → Strong for multimodal AI
- **Vespa** → Large-scale serving and ranking
- **Vald** → Kubernetes-native distributed search
- **Azure AI Search** → Enterprise search with vector support

Choosing the right one:

- Prototype → Chroma, pgvector
- SaaS production apps → Pinecone, Weaviate, Qdrant
- Massive scale → Milvus, Vespa
- Existing PostgreSQL → pgvector
- Existing Redis → Redis Vector
- Existing MongoDB → Atlas Vector Search

Bottom line:

AI progress isn't just about better models.

It's about **better retrieval systems**.

And that starts with the right vector database.

The Problem With Legacy Recruitment Tools

Traditional recruitment systems are often limited by outdated keyword matching algorithms. This creates a significant "blind spot," where highly skilled candidates are overlooked if their profiles don't perfectly align with exact search terms.

The result is a narrower talent pool, slower hiring processes, and increased bias, preventing organisations from discovering the true potential within the global workforce.



The Problem With Legacy Recruitment Tools

Relying on keyword matching in 2026 is like finding a master chef by searching only for people who own a whisk. Legacy ATS platforms filter out brilliance by design.

✗ What Legacy Tools Do

- Exact keyword matching only
- Count years linearly, ignoring depth
- Drop candidates missing one keyword
- Ignore GitHub, portfolio, and community signals

✓ What Our Engine Does

- Understands skill adjacencies and context
- Evaluates career trajectory and velocity
- Boosts "hidden gems" when core concepts match
- Weighs real-world activity and recency



How Tech is Transforming Engagement

Practical shifts in behavior, channels, and measurement for modern audiences

presentations.ai

Three AI Pillars Powering the Engine

The engine moves from **lexical search** (exact words) to **semantic search** (meaning and intent) through three interconnected AI layers.



JD Deconstruction

Breaks every Job Description into **Mandatory**, **Nice-to-Have**, and **Implied** skills — so nothing important is missed.



Candidate Vectorization

Transforms chaotic resumes and activity signals into unified mathematical representations using dense embeddings.



Contextual Re-Ranking

Scores candidates on **predicted success** for the specific role — not a binary keyword match.

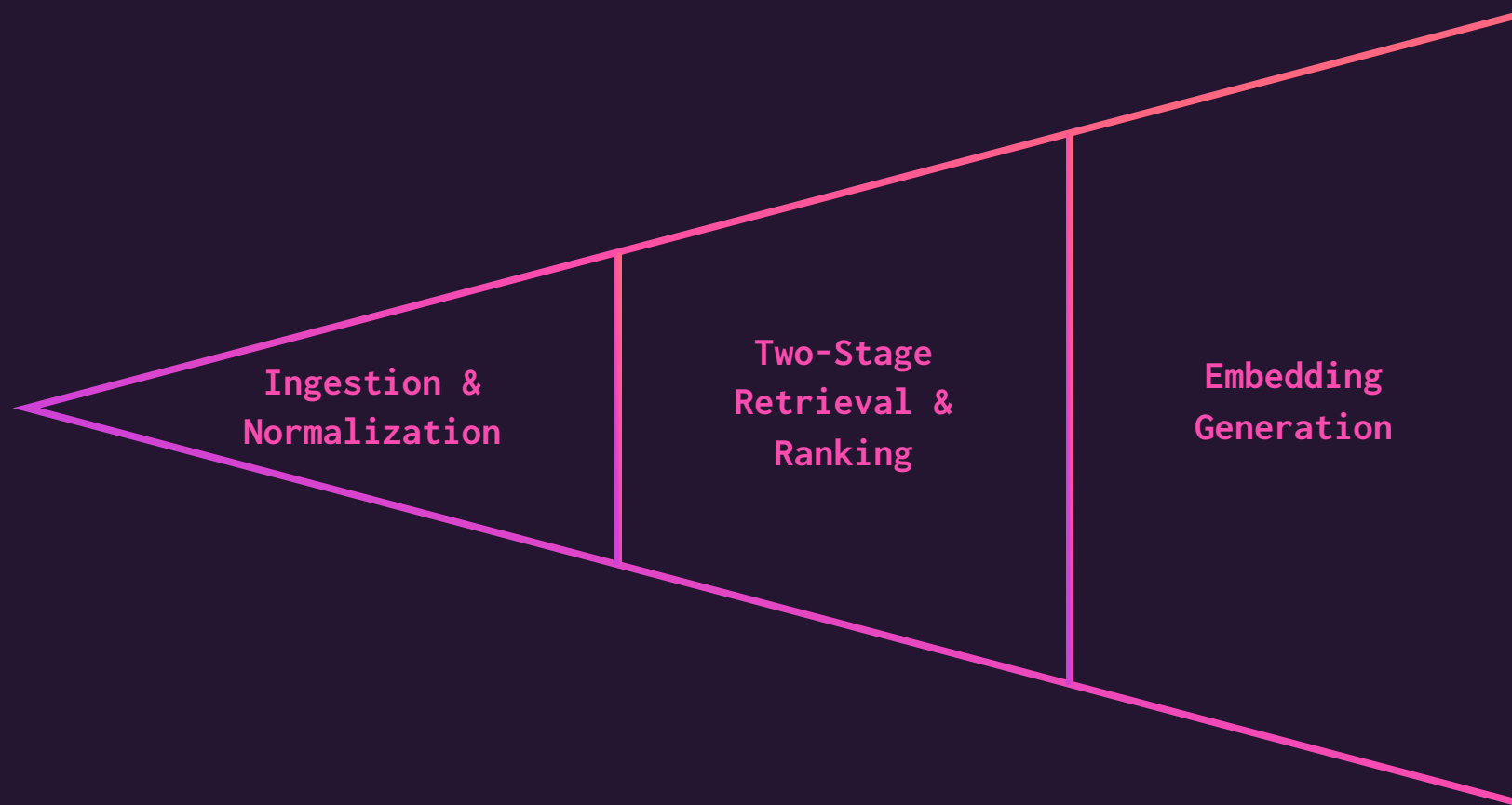
Intelligent Candidate Discovery

Semantic search, activity signals, and trajectory-aware ranking to surface fit and hidden talent faster than keyword ATS



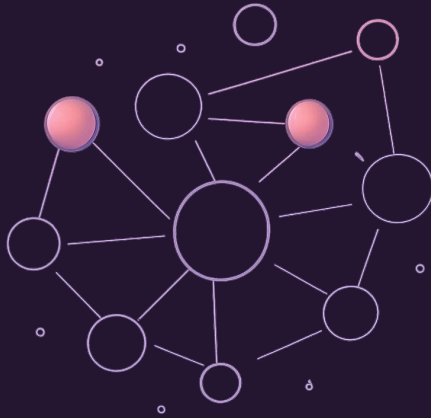
System Architecture: The Two-Stage Pipeline

Running a heavy LLM on millions of candidates per search is computationally impossible. The solution: a **fast retrieval stage** followed by a **deep-reasoning ranking stage**.



This architecture ensures millisecond-speed recall at scale, followed by precision ranking that understands career context — not just keyword overlap.

Stage 1: Vector Search Pipeline (Recall)



Dense Vector Search at Scale

We don't embed an entire resume as one block — that dilutes the signal. Instead, profiles are chunked into **Recent Experience**, **Core Skills**, and **Historical Trajectory**, then embedded using models like `text-embedding-3-small`.

HNSW Index

Multi-layered graph that skips full scans, zeroing in on closest vector clusters in milliseconds.

Cosine Similarity

"Engineered predictive models using Python" matches a JD asking for "Machine Learning Engineer" — because vectors map *meaning*, not spelling.

Stage 2: The Skill Knowledge Graph (Enrichment)

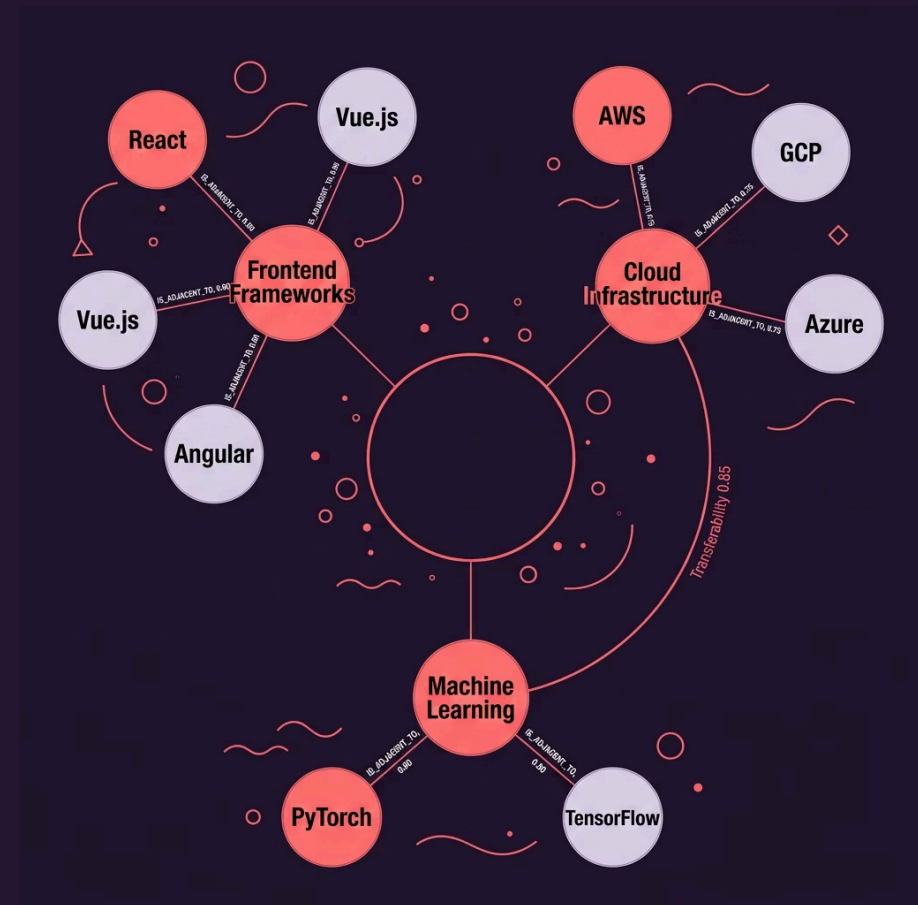
Structural Industry Intelligence

Vector search is powerful but lacks deterministic logic. The **Skill Knowledge Graph (SKG)** — built in Neo4j — gives the engine structured understanding of how skills, tools, and titles relate.

- **Nodes:** React, Vue.js, Frontend Developer, AWS, GCP
- **Edges:** [React] → *IS_ADJACENT_TO* → [Vue.js] (Weight: 0.85)
- **Auto-enrichment:** A candidate with React + Angular automatically inherits the parent node *Frontend Frameworks*



Key insight: The SKG mathematically proves a candidate with 4 years of GCP is fully capable of an AWS role — transferability score: **0.85**.



Stage 3: Contextual Re-Ranking (Precision)

The top 1,000 candidates enter the **Learning to Rank (LTR)** stage — scored 0–100 across four dimensions.

1

Semantic Score (S_{emb})

Cosine similarity from the vector database — textual meaning alignment.

2

Graph Match (S_{graph})

Structural overlap between JD requirements and the candidate's SKG nodes.

3

Trajectory Velocity (V_{traj})

Promotions over time — Junior to Senior in 3 years signals high velocity.

4

Recency Decay (D_{rec})

Exponential decay on skills — React used yesterday scores higher than 4 years ago.

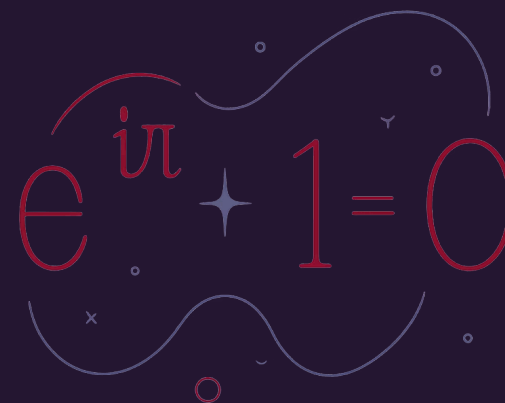
The Ranking Formula

Weighted Score Calculation

The final rank combines all four dimensions into a single, learnable score. Weights α and β are trained on historical hiring outcomes.

$$RankScore = \sum_{i=1}^n (\alpha S_{emb,i} + \beta S_{graph,i}) \times V_{traj} \times e^{-\lambda D_{rec}}$$

- α, β — learned weights for textual vs. structural match
- λ — controls how steeply historical skills lose relevance
- V_{traj} — amplifies fast risers; $e^{-\lambda D_{rec}}$ — discounts stale skills



The Hidden Gem

Algorithm: Trajectory Velocity

Brilliant candidates often have messy resumes. Our **Trajectory & Signal Modifier** surfaces them by measuring real growth — not just job titles.

⚡ Velocity Score

$$V_{traj} = \frac{Level_{current} - Level_{start}}{T_{active}}$$

Junior (2) → Senior (4) in
4 years = **0.5 levels/year**

🔧 Platform Activity

Consistent GitHub commits, portfolio work, and community contributions boost scores — even when titles don't perfectly match.

🌉 Skill Adjacency Bridge

JD asks for AWS, candidate has GCP. Legacy ATS drops them. Our SKG recognizes both as **Cloud Providers** with 0.85 transferability — keeping them in the running.

No Gap Penalties. No Unfair Demotions.

Active Time Calculation

Traditional systems divide progress by *chronological time* — a 2-year caregiving gap artificially tanks your velocity. Our engine uses **Active Time**:



→ **Pause the Clock**
Gaps for caregiving, health, or travel are **mathematically ignored** — preserving actual learning speed.

→ **Alternative Signal Catching**
GitHub commits or certifications during a gap are flagged as **Active Upskilling** — adding skill weight without penalty.

→ **Pivot Normalization**
Switching domains (e.g., Senior Backend → Junior ML) is treated as a **Pivot**, not a demotion — velocity resets for the new domain.